



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN



LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO
COMPUTADORA

EJERCICIOS DE CLASE N° 09

NOMBRE COMPLETO: Velazquez Caudillo Osbaldo

N° de Cuenta: 320341704

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 28/10/25

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

1.- Agregar que el número cambiante sea a una velocidad visible.

Declaramos las variables que se van a utilizar

```
//Ejercicio 1
double periodo_cambio_numero = 0.0;
int indice_numero = 0;
const int columnas_numeros = 4;
```

Inicializamos las variables

```
periodo_cambio_numero = 2.0;
```

```
if (now >= periodo_cambio_numero) {
    periodo_cambio_numero += 1.0;
    indice_numero = (indice_numero + 1) % columnas_numeros; // los numero del 1 al 4
}

//printf("paso_u_numeros %f\n", paso_u_numeros);

toffsetnumerocambiau = 0.25f * indice_numero; // 0, 0.25, 0.5, 0.75
toffsetnumerov = 0.0f; // misma fila (ajusta si tu atlas usa otra)

toffset = glm::vec2(toffsetnumerocambiau, toffsetnumerov);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-10.0f, 10.0f, -6.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
NumerosTexture.UseTexture();
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[6]->RenderMesh();
```

Usamos now que es una variable que se usa como reloj global y utilizo a periodo_cambio_numero como el siguiente instante en el que debo cambiar de número; cuando now lo alcanza, le sumo +1.0 para guardar el próximo cambio un segundos y avanzo indice_numero ciclando con % columnas_numeros (así recorro 1,2,3,4). Luego calculo el desplazamiento de la textura en U como 0.25f * indice_numero porque son 4 columnas (cada una

vale 0.25 del ancho), y dejo V en 0.0f porque sigo en la misma fila. Con esos dos valores armo toffset y se lo mando al shader.

Ejecución



2.- Alternar entre la textura del número 1 y 2

Declaramos las variables

```
//Ejercicio 2
double periodo_alterna_textura = 0.0f;
bool usa_textura1;
```

Inicializamos las variables

```
periodo_alterna_textura = 1.0f;
usa_textura1 = true; // empieza mostrando la textura 1
```

```

// printf("now> %f periodo_alterna_textura %f", now, periodo_alterna_textura);
if (now >= periodo_alterna_textura) {
    usa_textural = !usa_textural; // cambia 1 0 2
    periodo_alterna_textura += 1.0f; // programa el próximo cambio (mismo periodo)
}

offsetnumerou = 0.0;
offsetnumerov = 0.0;
offset = glm::vec2(offsetnumerou, offsetnumerov);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-13.0f, 10.0f, -6.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(offset));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Numero1Texture.UseTexture();

// elige la textura según la bandera
if (usa_textural) {
    Numero1Texture.UseTexture();
}
else {
    Numero2Texture.UseTexture();
}

Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

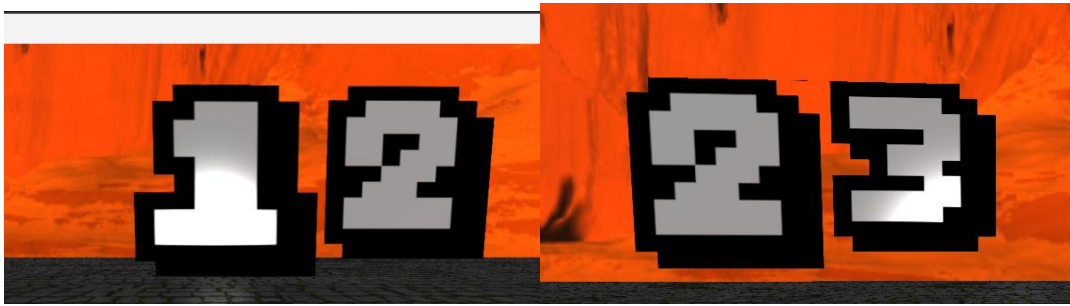
```

Para el cambio de texturas hacemos que dependa del tiempo real. Para eso uso now igual que en el ejercicio 1 y trato periodo_alterna_textura como el siguiente instante en el que debo alternar.

Como la variable usa_textura1 esta inicializada en true, va a iniciar con la textura1.

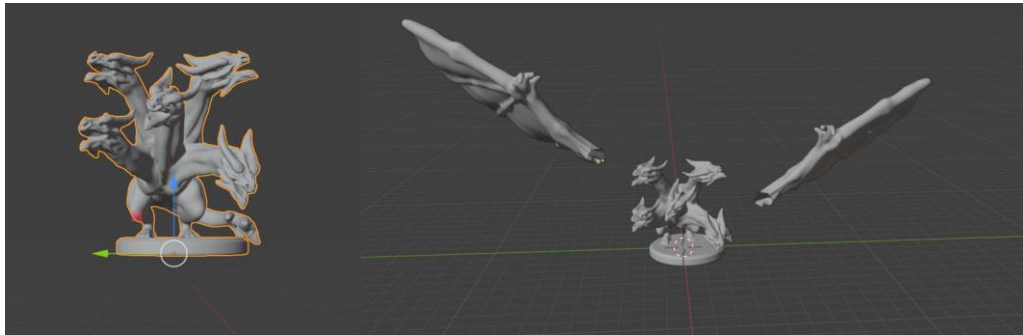
En el if lo que hacemos es verificar si now es mayor que es el valor de la variable periodo_alterna_textura, entonces usa_textura1 cambia de valor esto para que el if de hasta abajo revise el valor y alterne entre ambas. Y por último aumentamos el valor de periodo_alterna_textura en 1 para que se haga el siguiente cambio, entonces dicho de otra manera periodo_alterna_textura es el tiempo que nos va a durar la textura antes del cambio y también funciona para programar el siguiente cambio en segundos dentro del condicional.

Ejecución



3.- Separarle las alas al dragón, hacer que el dragón avance 20 unidades de forma senoidal aleteando y regrese volteado/ no de espaldas (loop).

Lo primero fue en Blender separar las alas.



Y exportamos los modelos para usarlos

Declaramos variables

```
//Ejercicio 3  
Model Dragon_Ala_D;  
Model Dragon_Ala_I;
```

Importamos los objetos en OpenGL

```
//Ejercicio3  
Dragon_M = Model();  
Dragon_M.LoadModel("Models/dragon.obj");  
Dragon_Ala_D = Model();  
Dragon_Ala_D.LoadModel("Models/Ala_derecha.obj");  
Dragon_Ala_I = Model();  
Dragon_Ala_I.LoadModel("Models/Ala_izquierda.obj");
```

Inicializamos variables

```

avanzadragon = true;
giro = 180 * toRadians;
posdragon = 1.0f;
mult_alateo = 1.0f;
max_ala_deg = 30.0f;
ang_ala_deg = 0.0f;
ang_ala = 0.0f;

```

Para hacer que el dragón tenga las alas a sus costados y que estén ahí mientras se mueva las agregamos por jerarquía

```

//dragonavance para moverlo hacia adelante y atrás en X
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f+ dragonavance, 5.0f + sin(glm::radians(angulovaria)), 6.0));
modelaux = model;
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, giro, glm::vec3(0.0f, 1.0f, 0.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
color = glm::vec3(0.0f, 1.0f, 0.0f);
//glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Dragon_M.RenderModel();

//Ala derecha
model = modelaux;
model = glm::translate(model, glm::vec3(-0.2f*posdragon, 0.96, 0.01*posdragon));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, giro, glm::vec3(0.0f, 1.0f, 0.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
/*color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));*/
model = glm::rotate(model, +ang_ala, glm::vec3(1, 0, 0));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Dragon_Ala_D.RenderModel();

```

```

//Ala izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(-0.2f*posdragon, 0.88, -0.15*posdragon));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, giro, glm::vec3(0.0f, 1.0f, 0.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
/*color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));*/
model = glm::rotate(model, -ang_ala, glm::vec3(1, 0, 0));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Dragon_Ala_I.RenderModel();

```

```

if (avanzadragon) {
    printf("dragonavance %f >", dragonavance);
    if (dragonavance > -20.0f) {
        dragonavance -= avanceoffset * deltaTime;
        printf("dragon avancen en el if: %f\n", dragonavance);
    }
    else {
        avanzadragon = !avanzadragon;
        giro = 0 * toRadians;
        posdragon = -1.0f;
    }
}
else {
    if (dragonavance < 20.0f) {
        dragonavance += avanceoffset * deltaTime;
    }
    else {
        avanzadragon = !avanzadragon;
        giro = 180 * toRadians;
        posdragon = 1.0f;
    }
}
}

```

Para hacer que el dragón recorra 20 unidades utilizo la bandera de avanzadragon inicializada en true y la posición dragonavance mientras

avanzadragon es verdadera, le resto a dragonavance (con avanceoffset * deltaTime) hasta llegar a -20.0; cuando toca ese límite, volteo el sentido (avanzadragon = !avanzadragon), le cambio la orientación con giro (de 180° a 0° o viceversa) y ajusto posdragon a ± 1 para que los offsets de las alas se reflejen correctamente al rotar el cuerpo(esto para que quede en la posición correcta ya que si no le ponía este valor las alas se movían de su lugar). En el regreso hago lo mismo, pero sumando hasta +20.0; así queda un loop ida y vuelta entre -20 y +20.

Para que las alas se muevan sale de otra senoide: calculo ang_ala_deg = sin(radians(angulovaria * mult_aleteo)) * max_ala_deg, lo paso a radianes y se lo aplico a cada ala con signo opuesto (+ang_ala a la derecha y -ang_ala a la izquierda) para que una suba mientras la otra baja.

```
ang_ala_deg = sin(glm::radians(angulovaria * mult_aleteo)) * max_ala_deg;
ang_ala = glm::radians(ang_ala_deg);
```

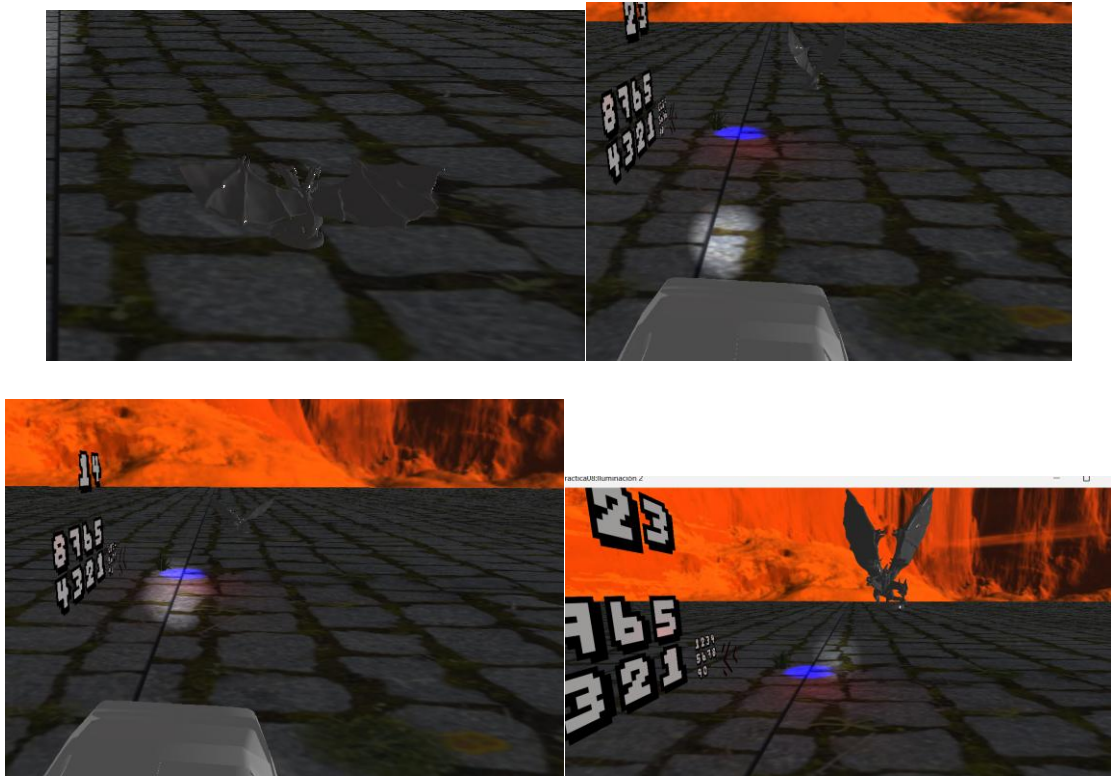
Entonces la variable giro se va a estar intercalando, dependiendo de la posición y es ese valor que le va a llegar a la rotación en cada una de las alas.

```
//Ala derecha
model = modelaux;
model = glm::translate(model, glm::vec3(-0.2f*posdragon, 0.96, 0.01*posdragon));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, giro, glm::vec3(0.0f, 1.0f, 0.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
/*color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));*/
model = glm::rotate(model, +ang_ala, glm::vec3(1, 0, 0));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Dragon_Ala_D.RenderModel();
```

```
//Ala izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(-0.2f*posdragon, 0.88, -0.15*posdragon));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, giro, glm::vec3(0.0f, 1.0f, 0.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
/*color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));*/
model = glm::rotate(model, -ang_ala, glm::vec3(1, 0, 0));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Dragon_Ala_I.RenderModel();
```


Ejecución



2. Problemas presentados. Listar si surgieron problemas a la hora de ejecutar el código

Un error que estaba ocurriendo fue con alas pues cuando hacia el cambio en la rotación las alas no se acomodaban en su lugar, esto se arregló haciendo una multiplicación por 1 o menos -1 dependiendo del caso solo así se pudo arreglar ese problema con las alas.

3. Conclusión:

Fue un ejercicio bastante practico de mucha lógica, me gusto, pero si considero que fue algo complicado de entender sobre todo lo de los números cambiantes del 1 al 4 y las alas del dragón, me parece que el nivel no es el adecuado para ser el primer acercamiento con la animación, pero por lo demás todo bien, creo que entendí de buena manera algunos conceptos fundamentales de la animación con la explicación de la sesión de laboratorio.